University of Science

Faculty of Information Technology

Data Structures and Algorithms Project Report

Topic: STRING MATCHING

Course: Data Structures & Algorithms

Instructor:
Le Thanh Tung
Tran Hoang Quan

Class: 25C10
Group ID: 07

Group Members:
Dang Ba Lam - 25127403
Hua Minh Khang - 25127073
Trinh Bao Khoi - 25127256
Nguyen Huu Ngoc Thuan - 25127153

Ho Chi Minh City, March 13, 2026

Contents

1 Group Information **2**

2 Problem Statement **3**

3 Algorithms Analysis **4**
3.1 The Naïve string-matching algorithm 4
3.2 The Rabin-Karp Algorithm 9
3.3 The Knuth-Morris-Pratt algorithm 14
3.4 The Boyer Moore Algorithm 20
3.5 The Aho-Corasick algorithm 25

4 Experimental Results **31**
4.1 Experimental Data 31
4.2 Evaluation 32
4.3 Graphs 34

5 Acknowledgements **36**

6 References **37**

1 Group Information

Student Name	**ID**	**Assigned Tasks**	**% Completed**
Dang Ba Lam	25127403	Leader, Implementation (The Brute Force algorithm, Data Generation), Report (The Brute Force algorithm, Problem Statement, Acknowledgement)	25%
Hua Minh Khang	25127073	Implementation & Report (The Knuth-Morris-Pratt algorithm, The Aho Corasick algorithm)	25%
Trinh Bao Khoi	25127256	Implementation (The Boyer-Moore algorithm, Command Line Parser and Output Formatting), Report (The Boyer-Moore algorithm), Recording the demonstration video	25%
Nguyen Huu Ngoc Thuan	25127153	Implementation (The Rabin-Karp algorithm, Executor), Report (The Rabin-Karp algorithm, Experimental Results)	25%
		Total Group Completion	**100%**

Table 1: Group Information

**Note: All members participated in Running Time and Comparisons table, implementation, Report writing, Citations, and Final checking.*

2 Problem Statement

STRING matching is a problem that find all occurrences of a pattern in the text. This problem is an indispensable technique in computer science for solving various crucial problems, such as text editing, text searching, and symbol manipulation.[5]

The string matching problem is defined as follows: We assume that the text is a string $T[1..n]$ of length n and the pattern is a string $P[1..m]$ of length m which $m \leq n$. The problem is to find all occurrences of P in T by shifting P along T such that $0 \leq s \leq n - m$ (s is the shift amount) and $T[s+1..s+m] = P[1..m]$ (that is, if $T[s+i] = P[i]$, for $1 \leq i \leq m$).[2]

For instance:

- If $T =$ "ababcab" and $P =$ "abc", then the output is $(3 \rightarrow 5)$ because the pattern occurs at position 3 in the text.

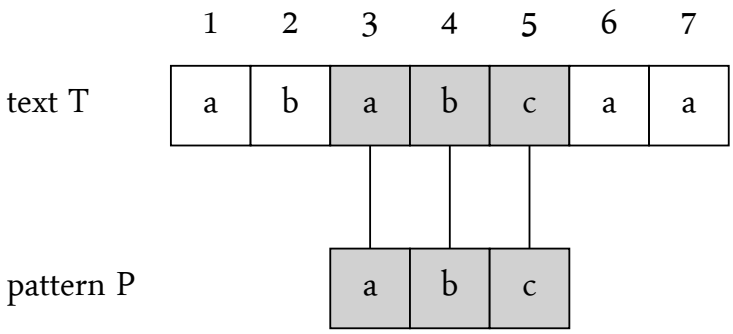


Figure 1: T = "ababcab" and P = "abc"

- If $T =$ "aaaaa" and $P =$ "aa", then the output is $(1 \rightarrow 2)$, $(2 \rightarrow 3)$, $(3 \rightarrow 4)$, and $(4 \rightarrow 5)$ because the pattern occurs at positions 1, 2, 3, and 4 in the text.

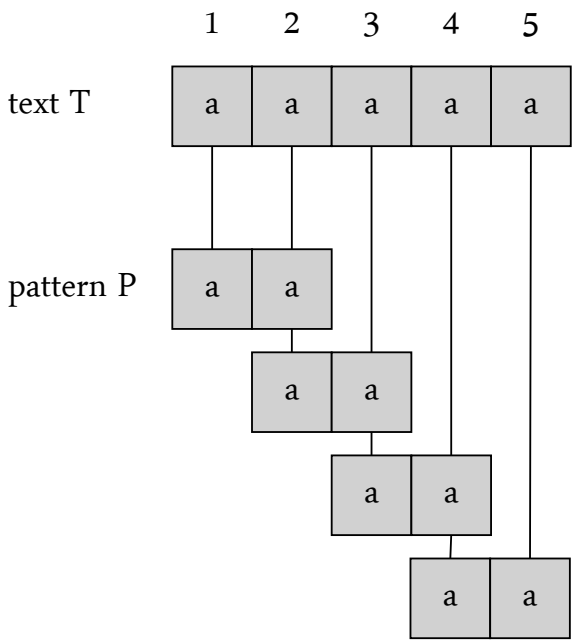


Figure 2: T = "aaaaa" and P = "aa"

3 Algorithms Analysis

3.1 The Naïve string-matching algorithm (Brute-Force approach)

3.1.1 Introduction & Application

- **Core idea:** According to Abdeen[1], the brute-force algorithm works with two pointers; a "text pointer" i and a "pattern pointer" j. For all possible shifts of the pattern P over the text T, compare the pattern characters with the corresponding text characters. If a mismatch occurs, i is incremented by 1, and j is reset to 0 and the comparing process is restarted. In the case that all characters match, the algorithm records the position of the match and continues searching for the next possible match.

- **Real-world applications:**
  - Text Editors: There are used in "Search" or "Find and Replace" features to locate target words.
  - Bioinformatics: Searching for specific DNA sequences.
  - Checking for Plagiarism: Comparing documents to find and compare a script or a piece of code with the exact match in a large database.

3.1.2 Pseudocode

Notation:

- **Text T (R rows, C columns)**: The main text being searched.
- **Pattern P (K: the number of keywords)**: The list of strings that we are looking for.
- **Positions ([(start position, end position), ...])**: A list storing the indices where a complete match is found.

```
BRUTE-FORCE-ALGORITHM(T, P, K, R, C)
pos = empty array of size K
for idx = 1 to K do
    keyword = P[idx]
    m = keyword.length

    ▷ Search Horizontally
    for i = 1 to R do
        for j = 0 to C − m do
            if keyword[1 ... m] == T[i, j + 1 ... j + m] then
                add position to pos[idx]
            end if
        end for
    end for

    ▷ Search Vertically
    for i = 1 to C do
        for j = 0 to R − m do
            if keyword[1 ... m] == T[j + 1 ... j + m, i] then
                add position to pos[idx]
```

```
            end if
        end for
    end for
end for
return pos
```

3.1.3 Step-by-step trace[2]

Notation:

- **Text** T **(length** n**)**: The main text being searched.
- **Pattern** P **(length** m**)**: The string that we are looking for.
- **Shift** (s): The current offset of the text T.
- **Positions ([(start position, end position), . . .])**: A list storing the indices where a complete match is found.

Example: text $T = [a, a, b, c, a, b, b]$ and pattern $P = [a, b, c]$

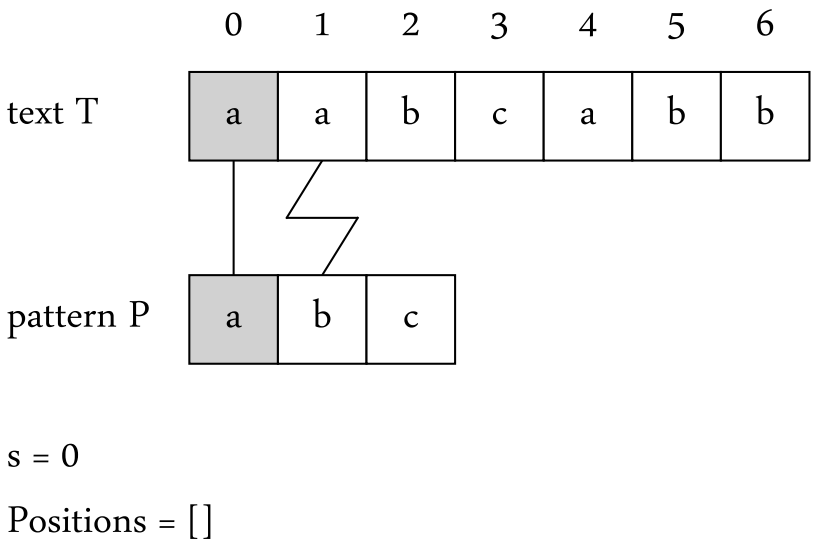
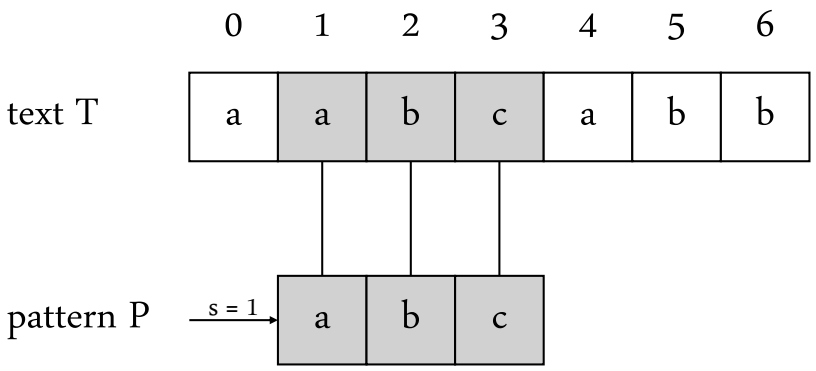


Figure 3: Step 1

- The first character 'a' matches between the pattern and the text.

  The second character in text T ('a') does not match the second character in pattern P ('b').

  Text at the bottom shows the current shift is $s = 0$ and the list of found matches is empty ($Positions = []$).



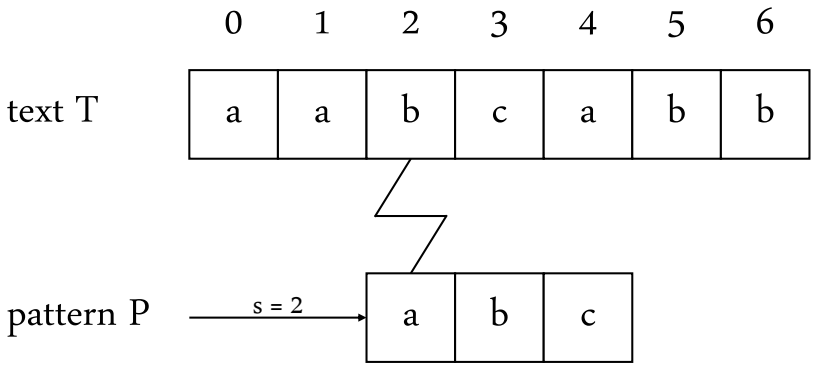
s = 1

Positions = [(1, 3)]

Figure 4: Step 2

- All characters in pattern P ('a', 'b', 'c') successfully match the corresponding characters in text T ('a', 'b', 'c').

  The shift is $s = 1$ and the list of found matches is updated to record this position ($Positions = [(1, 3)]$).



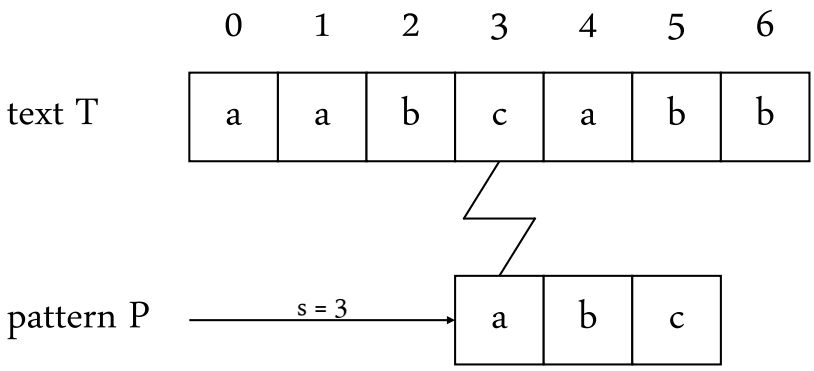
s = 2

Positions = [(1, 3)]

Figure 5: Step 3

- The first character in pattern P ('a') does not match the character in text T ('b') at the current shift position $s = 2$.

  Skip to the next shift without updating *Positions*.



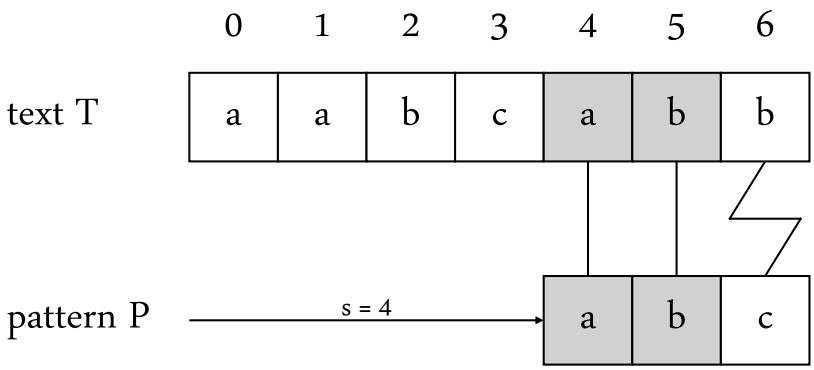
s = 3

Positions = [(1, 3)]

Figure 6: Step 4

- The first character in pattern P ('a') does not match the character in text T ('c') at the current shift position $s = 2$.

  Skip to the next shift without updating *Positions*.



s = 4

Positions = [(1, 3)]

Figure 7: Step 5

- The first two characters in pattern P ('a', 'b') match the corresponding characters in text T, but the third character in pattern P ('c') does not match the character in text T ('b') at the current shift position $s = 4$.

  Skip to the next shift without updating *Positions*.

- After that, s increases by 1. Because $s = 5$ and $n - m = 7 - 3 = 4$, we have $s > n - m$. The algorithm terminates and returns the list of found matches: Positions $= [(1, 3)]$.

3.1.4 Complexity

Metric	Complexity	Reasoning
Time (Average)	$O(K \cdot R \cdot C)$	In practice, a mismatch usually occurs after only a few character comparisons.
Time (Best with flag optimization)	$O(K \cdot R \cdot C)$	The first character of the pattern always results in a mismatch at every shift.
Auxiliary Space	$O(1)$	The algorithm uses only a constant amount of extra memory for loop indices.

Table 2: Complexity Analysis of 2D Brute Force Algorithm

3.1.5 Discussion

- In the basic brute-force implementation, the time complexity remains consistent across different scenarios.
- However, there is a way to optimize the brute-force algorithm by skipping some shifts when a mismatch occurs. This can be achieved by using a flag.
  - In Scenario 1, the number of patterns K is small, and the text is from 10×10 to 500×500. Brute-force approach is efficient and likely linear running time $O(R \cdot C)$.
  - In Scenario 2, the number of patterns K is large, and the text is fixed at 100×100. This scenario is more challenging for the brute-force approach, as the running time multiplies by K times, which is much longer compared to scenario 1.
  - In conclusion, the brute-force approach is more suitable for scenarios with a small number of patterns. It is worse for scenarios with a large number of patterns.

3.2 The Rabin-Karp Algorithm [3]

3.2.1 Introduction & Application

- **Core idea**: The Rabin-Karp algorithm uses a rolling hash function to find occurrences of a pattern P within a text T. Instead of comparing the pattern character by character for all possible shifts, it calculates a hash value for the pattern and a hash value for each window of the text. If the hash values match, it performs a character-by-character comparison to verify if it is a true match or a hash collision. The key advantage is that the hash value of the next window can be computed in $\mathcal{O}(1)$ time by dropping the first character and appending the new character.
- **Real-world applications**:
  - Plagiarism Detection: Comparing documents to find exact matches of sentences or paragraphs in a large database of academic papers.
  - Bioinformatics: Searching for specific DNA sequences within massive genetic codes efficiently.

3.2.2 Pseudocode

Notation:

- **Text T (R rows, C columns)**: The 2D grid of characters.
- **Pattern P (K keywords)**: The list of patterns to search for.
- **Modulo q**: A large prime number used to prevent integer overflow and minimize hash collisions (e.g., $q = 1000000007$).
- **Base d**: The number of characters in the alphabet (e.g., $d = 256$ for ASCII).

```
RABIN-KARP-2D-MATCHER(T, P, d, q)
R = rows[T], C = cols[T], K = count[P]
for p_idx = 0 to K − 1 do
    pat = P[p_idx]
    M = length[pat]
    h = d^(M−1) mod q
    p_hash = hash of pat
      ▷ Horizontal Search
    if M ≤ C then
        for r = 0 to R − 1 do
            t_hash = hash of T[r][0..M − 1]
            for c = 0 to C − M do
                if p_hash == t_hash and pat == T[r][c..c + M − 1] then
                    append (r, c) to Positions
                end if
                if c < C − M then
                    t_hash = (d · (t_hash − T[r][c] · h) + T[r][c + M]) mod q
                    if t_hash < 0 then
                        t_hash = t_hash + q
                    end if
                end if
```

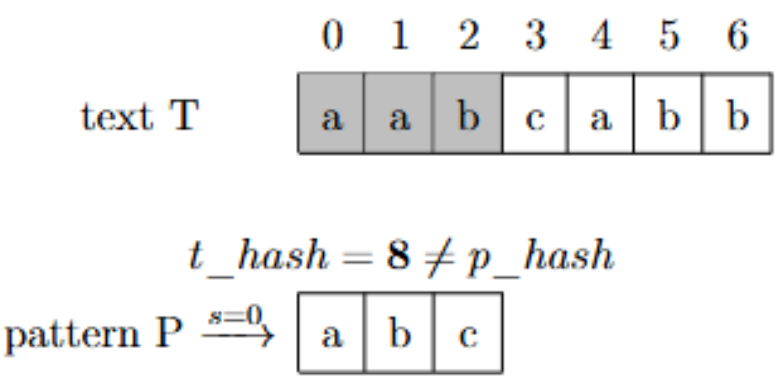
```
            end for
        end for
    end if
    ▷ Vertical Search
    if M ≤ R then
        for c = 0 to C − 1 do
            t_hash = hash of T[0..M − 1][c]
            for r = 0 to R − M do
                if p_hash == t_hash and pat == T[r..r + M − 1][c] then
                    append (r, c) to Positions
                end if
                if r < R − M then
                    t_hash = (d · (t_hash − T[r][c] · h) + T[r + M][c]) mod q
                    if t_hash < 0 then
                        t_hash = t_hash + q
                    end if
                end if
            end for
        end for
    end if
end for
```

3.2.3 Step-by-step trace (1D Example for Simplicity)

Notation:

- **Text** T **(length** n**)**: The main text being searched.
- **Pattern** P **(length** m**)**: The string that we are looking for.
- **Shift** (s): The current offset of the text T.
- **Positions** $([s_1, s_2, \dots])$: A list storing the indices where a complete match is found.

Example: text $T = [a, a, b, c, a, b, b]$ and pattern $P = [a, b, c]$.
For simplicity, we map characters to integers ($a = 1, b = 2, c = 3$), set base $d = 10$, and prime $q = 13$.
Hash of pattern P: $p_hash = (1 \cdot 10^2 + 2 \cdot 10 + 3) \bmod 13 = 123 \bmod 13 = \mathbf{6}$.



- The current text window is $T[0..2] = [a, a, b]$. The calculated hash is $t_hash = (1 \cdot 10^2 + 1 \cdot 10 + 2) \bmod 13 = 112 \bmod 13 = \mathbf{8}$.
- Since $t_hash \neq p_hash$ $(8 \neq 6)$, there is no need to compare characters.
- Text at the bottom shows the current shift is $s = 0$ and the list of found matches is empty ($Positions = []$).

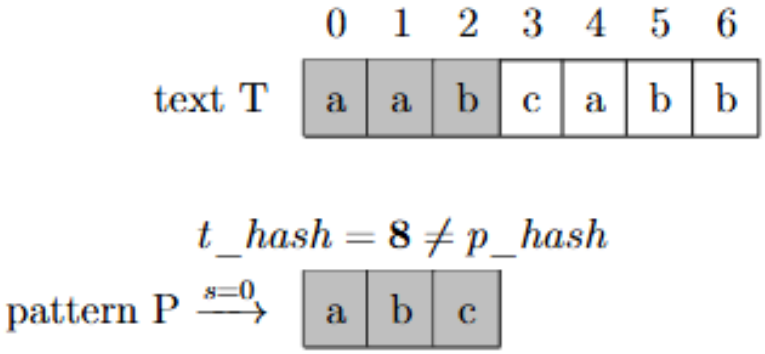


Figure X: Step 1

- The rolling hash is updated in $\mathcal{O}(1)$ time. The new window is $T[1..3] = [a, b, c]$ with $t_hash = 6$.
- Since $t_hash == p_hash$ $(6 == 6)$, the algorithm performs a secondary character-by-character check. All characters match successfully.
- Shift is $s = 1$ and the list of found matches is updated to record this position ($Positions = [1]$).

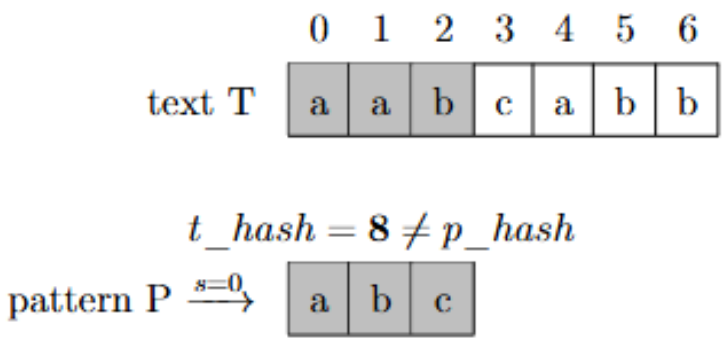


Figure X: Step 1

- The current text window shifts to $T[2..4] = [b, c, a]$.
- The rolling hash is updated in $\mathcal{O}(1)$ time. The new calculated hash is $t_hash = \mathbf{10}$.
- Since $t_hash \neq p_hash$ $(10 \neq 6)$, there is no need to compare characters.

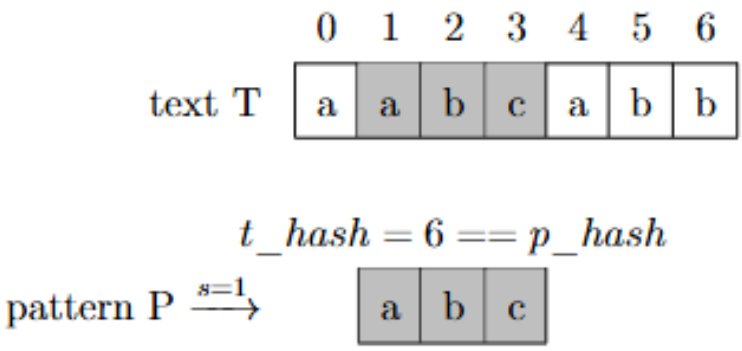


Figure Y: Step 2

- The current text window shifts to $T[3..5] = [c, a, b]$.
- The rolling hash is updated. The new calculated hash is $t_hash = \mathbf{0}$.
- Since $t_hash \neq p_hash$ $(0 \neq 6)$, there is no match.

3.2.4 Complexity

Metric	Complexity	Reasoning
Time (Worst)	$\mathcal{O}(K \cdot m \cdot R \cdot C)$	Occurs when the hash function generates frequent collisions (e.g., text and patterns share many identical substrings or all 'A's), forcing a full character comparison at every shift.
Time (Average)	$\mathcal{O}(K \cdot R \cdot C)$	In practice, with a strong hash function and large prime q, collisions are rare. The hash updates in $\mathcal{O}(1)$ time, skipping most character comparisons.
Time (Best)	$\mathcal{O}(K \cdot R \cdot C)$	The hash values never match (except on actual occurrences). The algorithm glides through the text in constant time per shift.
Auxiliary Space	$\mathcal{O}(1)$	The algorithm uses only a constant amount of extra memory for storing hash values and loop indices (excluding the space needed to store the output results).

Table 3: Complexity Analysis of 2D Rabin-Karp Algorithm

3.2.5 Discussion

- In **Scenario 1** (Impact of Grid Size), where the number of patterns K is small and the grid scales from 10×10 to 500×500, the Rabin-Karp algorithm is highly efficient. Due to its $\mathcal{O}(1)$ rolling hash update, the running time behaves almost linearly, similar to $\mathcal{O}(R \cdot C)$.
- In **Scenario 2** (Impact of Dictionary Size), where the grid is fixed at 100×100 and K increases, our specific implementation loops through the grid separately for each pattern. Therefore, the execution time scales linearly with K. While this is faster than Brute-Force in terms of raw character comparisons, it does not fully exploit Rabin-Karp's multi-pattern matching potential (which could theoretically search all patterns simultaneously using a unified hash table).
- **In conclusion**, the Rabin-Karp algorithm provides a significant speedup over Brute-Force by using mathematical hashing to filter out unnecessary comparisons. It is an excellent choice when dealing with large texts, provided a robust prime number q is chosen to avoid worst-case hash collision degradation.
- **Further Reading:** For a more rigorous mathematical proof regarding the probability of false positive hash collisions, please refer to the primary literature cited in Section 6.

3.3 The Knuth-Morris-Pratt algorithm (KMP algorithm)

3.3.1 Introduction & Application

- **Core idea:** The Knuth-Morris-Pratt (KMP) algorithm is a finite automaton which utilizes a prefix array (also known as the π). For each substring $P[0 \ldots \text{currentIndex}]$, the algorithm determines the longest proper prefix that is equal to a suffix of that same substring. The construction of this array is an iterative process based on prior states. When a mismatch occurs at the current index, the algorithm does not restart from the beginning; instead, it recursively consults previous prefix values until a matching character is found or the process reverts to the base state (0).
- **Real-world applications:**
  - Text Editors: Using in "Search" or "Find and Replace" features to locate destination words.
  - Bioinformatics: Searching for specific DNA.
  - Checking for Plagiarism: Comparing documents to find and compare a script or a piece of code with the exact match in a large database.

3.3.2 Pseudocode

Notation:

- **Text T (R rows, C columns)**: The 2D grid of characters.
- **Pattern P (K keywords)**: The list of patterns to search for.
- **Prefix Array π**: An array storing the lengths of the longest proper prefix that is also a suffix.

```
CONVERT-GRID-TO-STRINGS(T, R, C)
ans = empty array
    ▷ Extract Rows
for i = 0 to R − 1 do
    tmpString = empty string
    for j = 0 to C − 1 do
        tmpString = tmpString + T[i][j]
    end for
    append tmpString to ans
end for
    ▷ Extract Columns
for j = 0 to C − 1 do
    tmpString = empty string
    for i = 0 to R − 1 do
        tmpString = tmpString + T[i][j]
    end for
    append tmpString to ans
end for
return ans
```

```
CALCULATE-POSITION(idxString, m, R, idx)
if idxString < R then
```

```
    ▷ Horizontal match (Row extraction)
      start = (idxString, idx − 2 · m)
      end = (idxString, idx − m − 1)
else
    ▷ Vertical match (Column extraction)
      start = (idx − 2 · m, idxString − R)
      end = (idx − m − 1, idxString − R)
end if
return (start, end)
KMP-Algorithm(T, P, K, R, C)
pos = array of K empty lists
List_string = Convert-Grid-To-Strings(T, R, C)
N = List_string.length

for idxString = 0 to N − 1 do
    for pIdx = 0 to K − 1 do
        kmpString = P[pIdx] + '#' + List_string[idxString]
        L = kmpString.length
        m = P[pIdx].length
        π = array of size L initialized to 0
    ▷ Compute Prefix Function & Search
        for i = 1 to L − 1 do
            j = π[i − 1]
            while j > 0 and kmpString[j] ≠ kmpString[i] do
                j = π[j − 1]
            end while
            if kmpString[j] == kmpString[i] then
                j = j + 1
            end if
            π[i] = j
    ▷ Match found
            if π[i] == m then
                matchPos = Calculate-Position(idxString, m, R, i)
                append matchPos to pos[pIdx]
            end if
        end for
    end for
end for
return pos
```

3.3.3 Step-by-step trace[2]

Notation:

- **Text** T **(length** n**)**: The main text being searched.
- **Pattern** P: The string that we are looking for.
- **Shift** (s): The current offset of the text T.

- **Positions ([(start position, end position), ...])**: A list storing the indices where a complete match is found.

Example: text $T = [b, a, n, a, b, a]$, pattern $P = [b, a, n, a]$ and string $S = [b, a, n, a, \#, b, a, n, a, b, a]$
Note: String S is the concatenation of the pattern, the sentinel character #, and the text. Any character can be used instead of #, provided it does not appear in T or P.

	0	1	2	3	4	5	6	7	8	9	10
string S	b	a	n	a	#	b	a	n	a	b	a
π array	0	0	0	0	0	-1	-1	-1	-1	-1	-1

Figure 8: Step 1

- Assuming we have calculated the π array up to index 6, while the remaining indices are set to -1 (indicating they have not yet been calculated).
  Note that $\pi[0]$ is always 0 because a single-character string has no proper prefixes.

	0	1	2	3	4	5	6	7	8	9	10
string S	b	a	n	a	#	b	a	n	a	b	a
Current substring	b										
π array	0	0	0	0	0	1	-1	-1	-1	-1	-1

Figure 9: Step 2

- To find the answer for index 5, we must look at the previous result at $\pi[4]$ first. Because $\pi[4]$ is 0, it tells us that there are no smaller matches to check, so we can stop looking back immediately. Now, we only need to compare $S[5]$ with the very first letter $S[0]$. Since both are 'b', they match, and we add 1 to get $\pi[5] = 1$. The calculation of the π array depends only on the array itself, and character comparisons are performed only when necessary.
  Note: To make this process easier to understand, the author provides an illustration showing the comparison between the current prefix and suffix at each step.

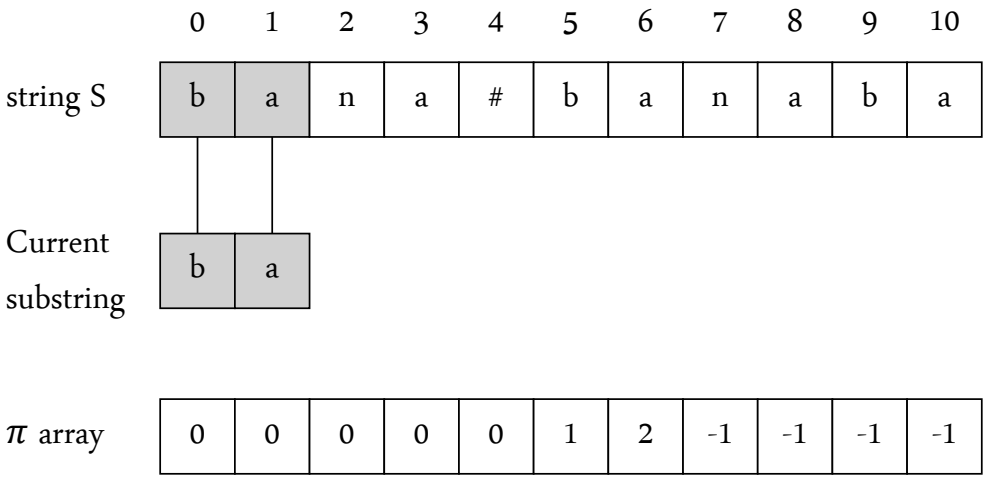


Figure 10: Step 3

- Similar to step 2, we look at the previous result at $\pi[5]$ first. We see that $S[6]$ matches $S[1]$ (both are 'a'), so the search stops there. Because they match, we increase the value from 1 to 2; therefore, $\pi[6] = 2$.

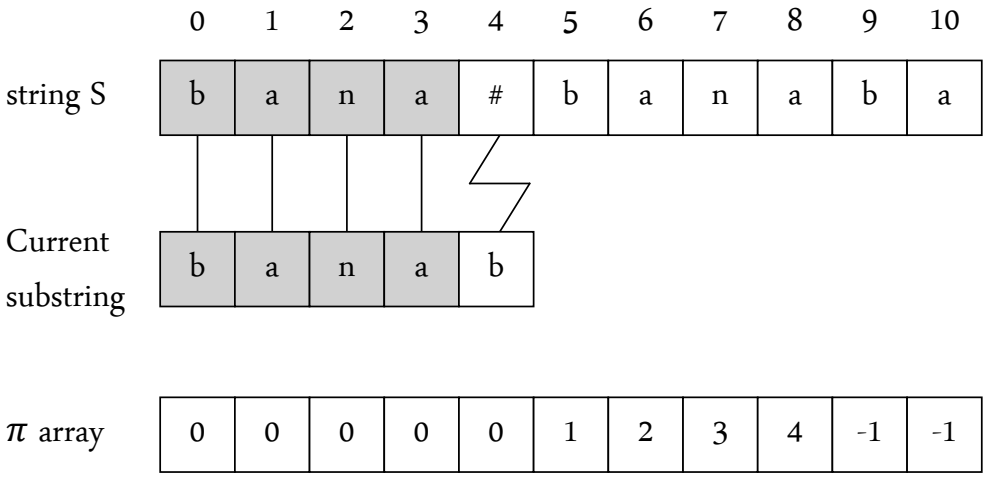


Figure 11: Step 4

- We skip to index 9 because indices 7 and 8 follow the same logic as indices 5 and 6 in the previous step. Now, we continue with index 9. We do the same steps as before and get a mismatch, because $S[9]$ is 'b' but $S[4]$ is $'\#'$. According to the KMP algorithm, we find the next possible position until it matches with $S[9]$ or the value reaches 0.

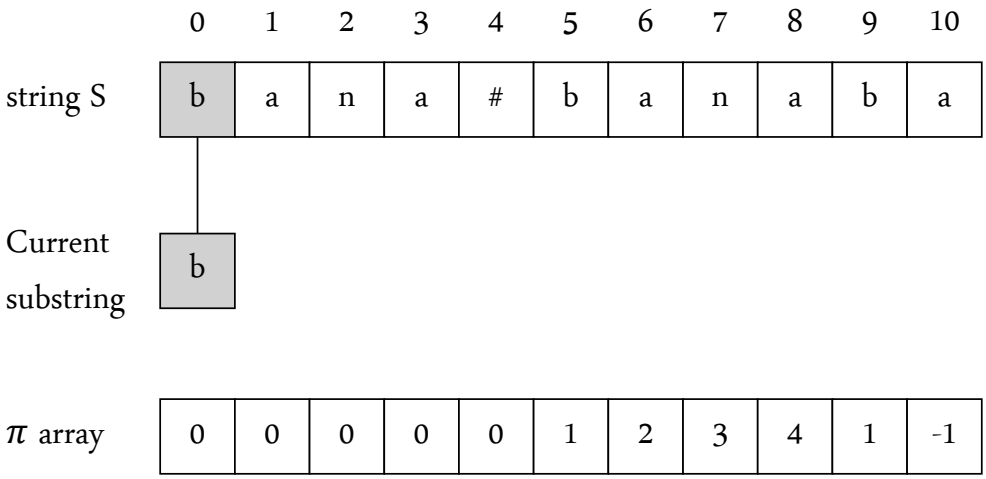


Figure 12: Step 5

- Following the KMP algorithm's instruction, we try with $\pi[current_value]$, which is $\pi[4]$. This gives us a value of 0, so we stop there. After finding the position, we must check whether that position matches the current index and we increase the value by 1 if it is a match. Fortunately, $S[9]$ matches $S[0]$, so $\pi[9] = 1$.

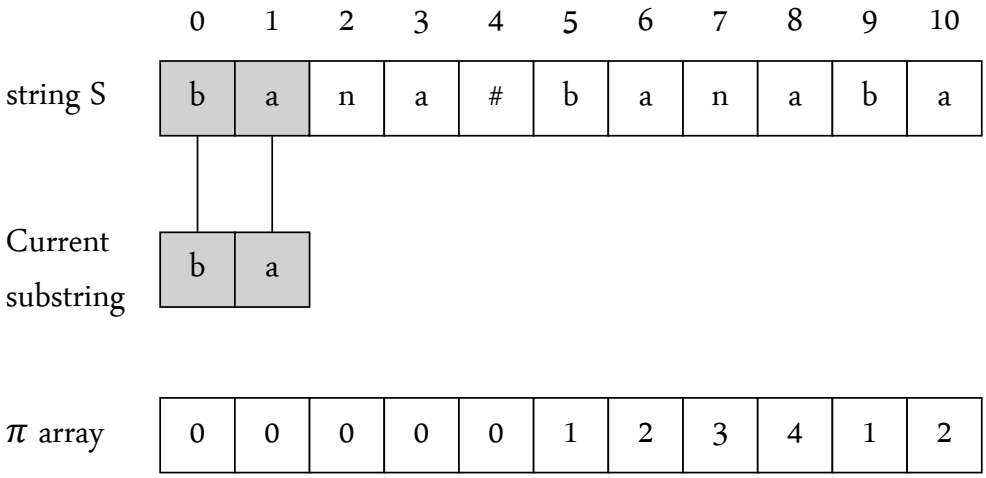


Figure 13: Step 6

- In the same way as the previous step, we take the value from the last index 9, which is 1. Because $S[10]$ matches $S[1]$, the algorithm stops looking back right now. Moving to the comparison step, since we know $S[10]$ and $S[1]$ match, we increase the value from 1 to 2, resulting in $\pi[10] = 2$.
  We have now traversed to the end of array S, and the algorithm terminates.
  Based on the π array, we find that there is 1 substring from the text T that matches pattern P by counting how many indices have a value equal to the length of P. The matching substring starts at index 5 (index 8 minus the length of P plus 1) and ends at index 8.

3.3.4 Complexity

Metric	Complexity	Reasoning
Time (Average)	$O(K \cdot (R + C) \cdot (m + D))$	The Knuth-Morris-Pratt algorithm is a finite automaton that finds all matching positions. Although the process of finding the next match may take several steps at some indices, it will take at most $2n$ comparisons in total. This means the Big O is $O(n)$ and the runtime is very stable. Note: $D = \max(R, C)$.
Auxiliary Space	$O(m + D)$	The algorithm uses extra memory for the prefix array (π table). Note: $D = \max(R, C)$.

Table 4: Complexity Analysis of 2D Knuth-Morris-Pratt Algorithm

3.3.5 Discussion

- In scenario 1, the number of patterns K is small, and the grid size is from 10×10 to 500×500. The Knuth-Morris-Pratt algorithm is an efficient algorithm for this scenario because it ranks third among the five fastest algorithms.
- In scenario 2, the number of patterns K is large, and the text is fixed at 100×100. In this case, the Knuth-Morris-Pratt algorithm takes more time to run because KMP is better for problems that don't have too many patterns to find. So, it takes the second rank of the slowest algorithms in this test.
- Although scenario 1 has a larger theoretical time complexity than scenario 2, scenario 2 consumes more time in practice. This is because the process of concatenating the grid strings and patterns must be done 20,000 times ($100 \cdot 100 \cdot 2$), while in case 1 it is only 5,000 times ($500 \cdot 2 \cdot 5$).
- In conclusion, the Knuth-Morris-Pratt algorithm is a suitable algorithm for string-matching, but it runs more efficiently when there are fewer patterns to find.

3.4 The Boyer Moore Algorithm[4]

3.4.1 Introduction & Application

- **Core idea:** The Boyer Moore Algorithm is a practical method to find patterns. This algorithm is the combination of three main ideas comprising scanning the pattern in reverse order, bad character heuristic and good suffix heuristic. This speeds up the searching process considerably by skipping a plenty of characters.
- **Reality applications:**
  - Text Editors: Using in "Search" or "Find and Replace" features to locate the destination of the words.
  - Intrusion Detection Systems (Network Security): use Boyer-Moore to scan network traffic in real-time for "malicious signatures"—specific byte patterns that indicate a cyberattack or virus.
  - Large-Scale Plagiarism Detection: compare submitted essays against billions of web pages and academic journals to find copied content.

3.4.2 Pseudocode

Notation:

- **Text T (R rows, C columns)**: The 2D character matrix.
- **Pattern P (K keywords)**: The list of strings to search.
- **Heuristics**: BC (Bad Character table) and GS (Good Suffix table).

```
BOYER-MOORE ALGORITHM(T, P, K, R, C)
pos = empty array of size K
for idx = 1 to K do
    P_i = P[idx], m = P_i.length
    BC = PreProcessBadChar(P_i)
    GS = PreProcessGoodSuffix(P_i)
  ▷ Horizontal Search (Rows)
    for r = 0 to R − 1 do
        s = 0
        while s ≤ C − m do
            j = m − 1
            while j ≥ 0 and P_i[j] == T[r, s + j] do
                j = j − 1
            end while
            if j < 0 then
                add (r, s) to pos[idx]
                s = s + GS[0]
            else
                s = s + max(GS[j], j − BC[T[r, s + j]])
            end if
        end while
    end for
  ▷ Vertical Search (Columns)
```

```
    for c = 0 to C − 1 do
        s = 0
        while s ≤ R − m do
            j = m − 1
            while j ≥ 0 and P_i[j] == T[s + j, c] do
                j = j − 1
            end while
            if j < 0 then
                add (s, c) to pos[idx]
                s = s + GS[0]
            else
                s = s + max(GS[j], j − BC[T[s + j, c]])
            end if
        end while
    end for
end for
return pos
```

3.4.3 Step-by-step trace

Notation:

- **Text** T **(length** n**)**: The main text being searched.
- **Pattern** P: The string that we are looking for.
- **Shift** (s): The current offset of the text T.
- **Positions ([(start position, end position), . . .])**: A list storing the indices where a complete match is found.

Example: text $T = [G, C, A, T, G, C, C, T, A, G]$ and pattern $P = [T, A, T, G]$

- Step 1: Build bad character and good suffix tables base on the pattern we are searching for:

char	A	T	G
shift value	1	2	3

Figure 14: Bad character table

index j	0	1	2	3
shift value	4	4	4	1

Figure 15: Good suffix table

Note:

- **Bad character table:** Choose the index for characters from 0 to n - 1. In this case, T appears two times, only the last appearance are regconized.
- **Good suffix table:** For index j from 3 to 0, if the suffix of the character at index j is found, the shift value will be decreased. In this example, index 3 is the exception since there is no suffix and the shift value is 1, other suffixes including g, tg, atg are not found in the prefix of the pattern, then their pattern shifts is the length of the pattern.

- Step 2: Compare the patterns and make shifts based on preprocessing values.

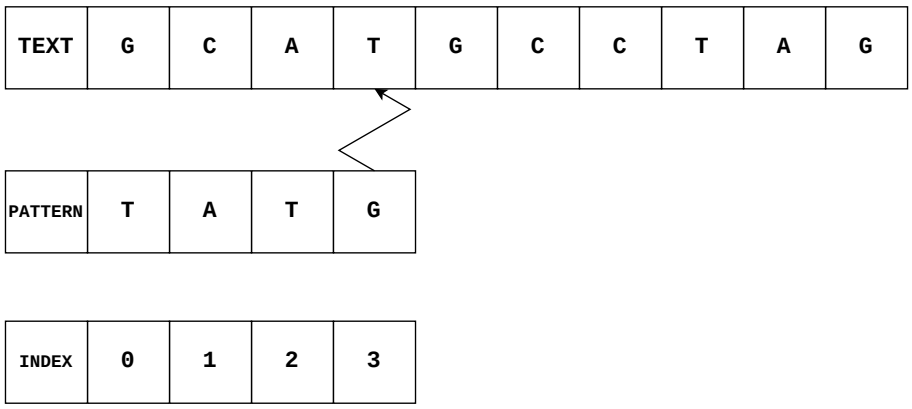


Figure 16: compare pattern from right to left

Mismatch at index j = 3, then calculate the shift value.

shift = max(GS, BC)

GS = 1, BC = max(1, j - lastOccur) = max(1, 3 - 2) = 1

⇒ shift value = 1

- **Continue searching process**

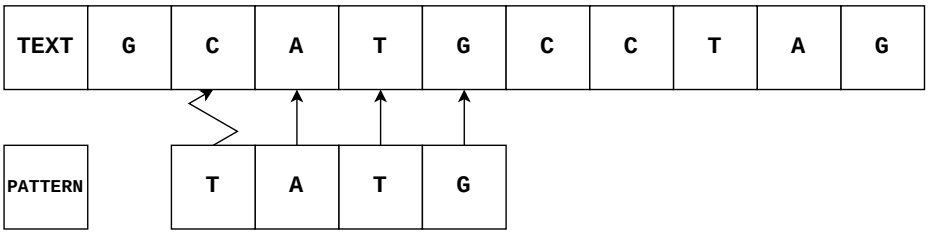


Figure 17: compare pattern

Mismatch at index 0, continue calculate the pattern shift
Shift value = max(GS, BC) = max(4, 1) = 4

- **Shift by 4 characters**

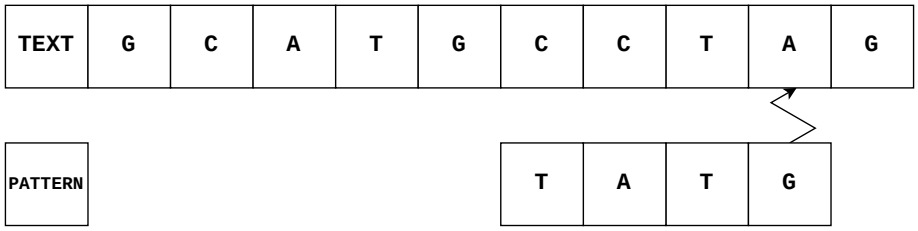


Figure 18: Make larger shift by using good suffix heuristic

Mismatch at index 3, continue calculate shift value
shift value = max(GS, BC) = max(1, 2) = 2

- **Shift by 2 characters**

Figure 19: Make bigger shift by using bad character heuristic

The algorithm stops since the pattern is out of text's range.

3.4.4 Complexity

Metric	Complexity	Reasoning
Time (Worst)	$O(K \cdot m \cdot R \cdot C)$	Occurs when many characters are repeated and both bad character and good suffix rules provide minimal shifts
Time (Average)	$O(R \cdot C)$	In reality, the rules of bad character and good suffix usually make larger skips and considerably improve the efficiency of the algorithm.
Time (Best)	$O(\frac{R \cdot C}{m})$	In best scenario, mismatches happen at the last character, allowing maximum shifts close to the pattern length(m).
Auxiliary Space	$O(\lvert\Sigma\rvert + m)$	The algorithm uses extra memory for bad character table (alphabet size) and good suffix table (pattern length)

Table 5: Complexity Analysis of 2D Boyer-Moore Algorithm

3.4.5 Discussion

- In Scenario 1, where the number of patterns K is small and the text size ranges from 10×10 to 500×500, the Boyer–Moore algorithm performs very efficiently due to its ability to skip large portions of the text. The running time is close to $O(R \cdot C)$ in practice.
- In Scenario 2, where the number of patterns K is large and the text is fixed at 100×100, the total running time increases linearly with K. However, Boyer–Moore still outperforms brute-force due to fewer comparisons per pattern.
- In conclusion, the Boyer–Moore algorithm is particularly effective when the pattern is relatively long and the alphabet is large, as it can achieve significant shifts and reduce the number of comparisons.

3.5 The Aho-Corasick algorithm

3.5.1 Introduction & Application

- **Core idea:** Aho-Corasick is a Trie with KMP failure links added in. It builds a Trie tree for all given patterns and links nodes to the longest proper suffix that is also a prefix in the Trie. If the algorithm hits a mismatch while processing, it just jumps to the next possible suffix by following the link instead of starting over. This keeps the search moving forward in one pass and if it reaches a output-node,and if it reaches an output-node, it identifies a pattern match within the text.
- **Real-world applications:**
  - Text Editors: Using in "Search" or "Find and Replace" features to locate destination words.
  - Bioinformatics: Searching for specific DNA.
  - Checking for Plagiarism: Comparing documents to find and compare a script or a piece of code with the exact match in a large database.

3.5.2 Pseudocode

Notation:

- **Trie Node**: A structure containing a character pC, a list of pattern indices pos, child pointers nxt, transition pointers go, a suffix link $link$, and a parent pointer par.
- **Text T (R rows, C columns)**: The 2D grid of characters.
- **Pattern P (K keywords)**: The list of patterns to search for.
- **List_string**: A 1D array of size $M = R + C$, containing all rows followed by all columns extracted from T.

```
ADD-STRING(S, idx, root)
cur = root
for each character c in S do
    if cur.nxt[c] == null then
        cur.nxt[c] = new Node
        cur.nxt[c].par = cur
        cur.nxt[c].pC = c
    end if
    cur = cur.nxt[c]
end for
append idx to cur.pos
```

```
GET-LINK(u, root)
if u.link == null then
    if u == root or u.par == root then
        u.link = root
    else
        u.link = GO(GET-LINK(u.par), u.pC)
    end if
end if
```

```
return u.link
GO(u, c, root)
if u.go[c] == null then
    if u.nxt[c] ≠ null then
        u.go[c] = u.nxt[c]
    else
        u.go[c] = (u == root) ? u : GO(GET-LINK(u), c)
    end if
end if
return u.go[c]
SEARCH-STRING(S, IDX, R, P, ans, root)
cur = root
for i = 0 to S.length − 1 do
    cur = GO(cur, S[i])
    tmp = cur
    while tmp ≠ root do
        for each idx in tmp.pos do
            m = P[idx].length
            if IDX < R then    ▷ Horizontal match (Row extraction)
                start = (IDX, i − m + 1)
                end = (IDX, i)
            else    ▷ Vertical match (Column extraction)
                start = (i − m + 1, IDX − R)
                end = (i, IDX − R)
            end if
            append (start, end) to ans[idx]
        end for
        tmp = GET-LINK(tmp)
    end while
end for
AHO-CORASICK(T, P, K, R, C)
List_string = CONVERT-GRID-TO-STRINGS(T)
ans = array of K empty lists
root = new Node
maxLen = max(R, C)
    ▷ Build the Trie
for i = 0 to K − 1 do
    if P[i].length ≤ maxLen then
        ADD-STRING(P[i], i, root)
    end if
end for
    ▷ Process all rows and columns
for i = 0 to List_string.length − 1 do
    SEARCH-STRING(List_string[i], i, R, P, ans, root)
end for

REMOVE-TRIE(root)    ▷ Free allocated memory
return ans
```

3.5.3 Step-by-step trace[2]

Notation:

- **Text** T **(length** n**)**: The main text being searched.
- **Pattern** P: The string that we are looking for.
- **Shift** (s): The current offset of the text T.
- **Positions ([(start position, end position), . . .])**: A list storing the indices where a complete match is found.

Example: text $T = [a, a, b, c, a, b, b]$ and pattern $P = [a, b, c]$

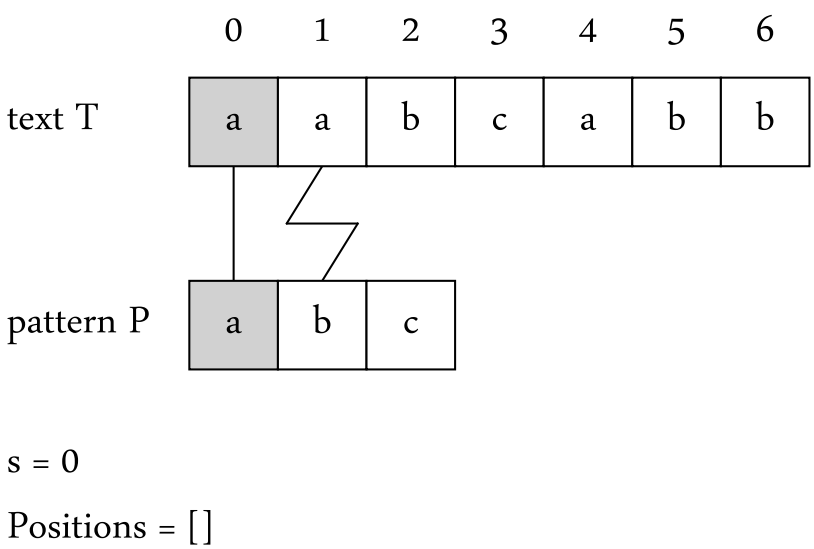
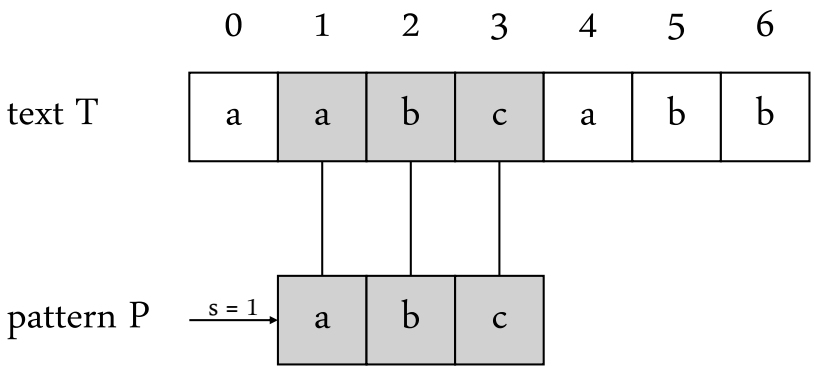


Figure 20: Step 1

- The first character 'a' matches in both arrays.

  The second character in text T ('a') does not match the second character in pattern P ('b').

  Text at the bottom shows the current shift is $s = 0$ and the list of found matches is empty ($Positions = []$).



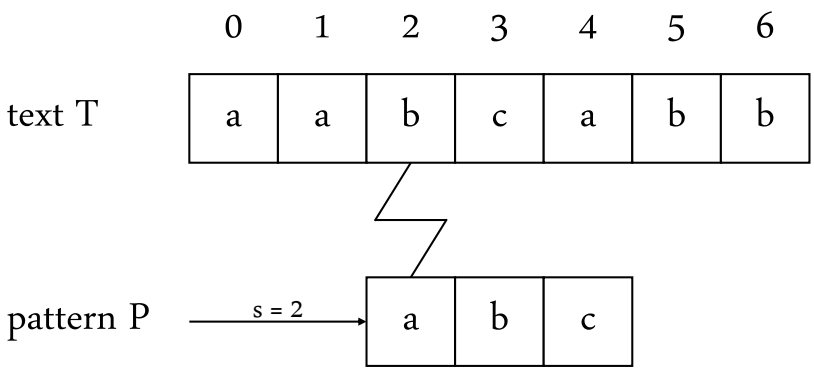
s = 1

Positions = [(1, 3)]

Figure 21: Step 2

- All characters in pattern P ('a', 'b', 'c') successfully match the corresponding characters in text T ('a', 'b', 'c').

  Shift is $s = 1$ and the list of found matches is updated to record this position ($Positions = [(1, 3)]$).



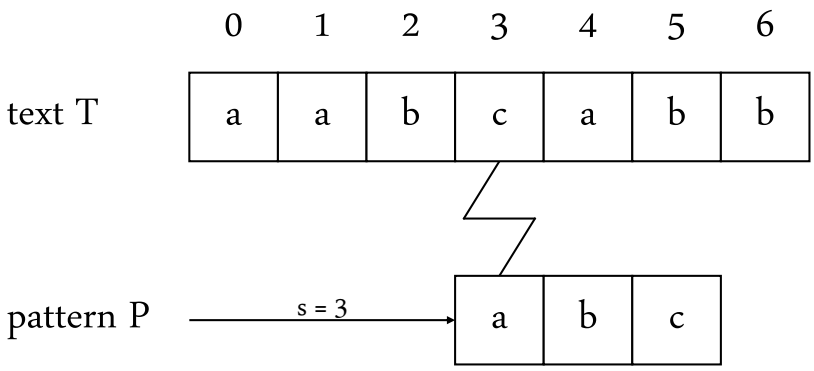
s = 2

Positions = [(1, 3)]

Figure 22: Step 3

- The first character in pattern P ('a') does not match the character in text T ('b') at the current shift position $s = 2$.

  Skip to the next shift without updating $Positions$.



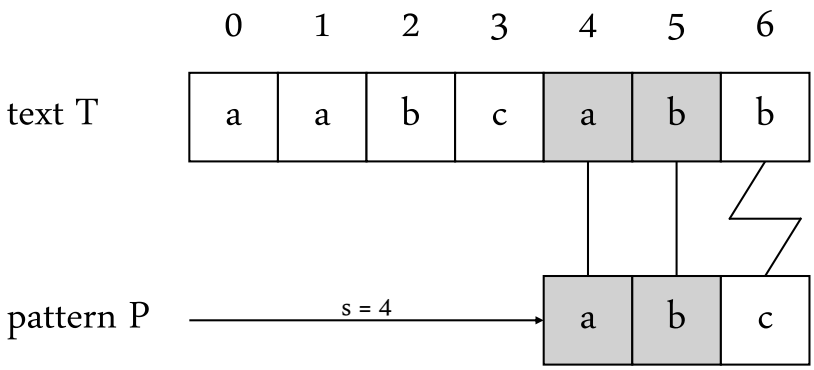
s = 3

Positions = [(1, 3)]

Figure 23: Step 4

- The first character in pattern P ('a') does not match the character in text T ('c') at the current shift position $s = 2$.

  Skip to the next shift without updating *Positions*.



s = 4

Positions = [(1, 3)]

Figure 24: Step 5

- The first two characters in pattern P ('a', 'b') match the corresponding characters in text T, but the third character in pattern P ('c') does not match the character in text T ('b') at the current shift position $s = 4$.

  Skip to the next shift without updating *Positions*.

- After that, s increases by 1. Because $s = 5$ and $n - m = 7 - 3 = 4$, we have $s > n - m$. The algorithm terminates and returns the list of found matches: Positions $= [(1, 3)]$.

3.5.4 Complexity

Metric	Complexity	Reasoning
Time (Worst)	$O(m \cdot R \cdot C)$	Without dictionary links, we must traverse failure links back to the root at every step. In the worst case (e.g., patterns are prefixes of each other), we might check m nodes for every character in the grid.
Time (Average)	$O(R \cdot C)$	In a random text, most steps do not find a match. This means we rarely need to climb the failure links, and the search stays near the root of the Trie.
Time (Best)	$O(R \cdot C)$	Occurs when characters in the grid never match any first character in the Trie, so the algorithm only does one comparison per index.
Auxiliary Space	$O(\sum \lvert P \rvert)$	The algorithm requires memory to store the Trie tree. The number of nodes is equal to the total length of all patterns added to the system.

Table 6: Complexity Analysis of Aho-Corasick Algorithm

3.5.5 Discussion

- In Scenario 1, the number of patterns K is small and the text size ranges from 10×10 to 500×500. The Aho-Corasick algorithm is not time-efficient in this case, and it takes second place among algorithms that consume the most time to run.
- In Scenario 2, the number of patterns K is large and the text is fixed at 100×100. This scenario is the opposite of the last case; when K becomes larger, Aho-Corasick's running time remains stable, which makes it the fastest algorithm as K increases.
- In conclusion, the Aho-Corasick algorithm is more suitable for scenarios where K increases over time. When there are many patterns to query, it is more efficient because it is usually used for cases with many string matching queries on a set of patterns.
- Besides that, in both scenarios, it does not rely on character comparisons, so it is efficient for string-matching problems with a high comparison cost.
- So the Aho-Corasick algorithm is one of the best solutions for this problem.

4 Experimental Results and Comments

4.1 Tables of Experimental Data

The following tables present the performance metrics (Running Time and Number of Comparisons) for five string matching algorithms evaluated under two different scenarios.

Note: The number of comparisons for the Aho-Corasick algorithm is recorded as 0 across all test cases. This is due to its underlying architecture; rather than performing traditional character-by-character comparisons, it traverses a pre-constructed finite state automaton (Trie with failure links), making traditional comparison tracking inapplicable.

Scenario 1: Varying Grid Size with Fixed Number of Keywords (5)

Grid Size	Keywords	Brute Force	Rabin-Karp	KMP	Boyer-Moore	Aho-Corasick
10×10	5	0.0739 ms	0.0569 ms	0.1547 ms	0.1110 ms	0.1718 ms
50×50	5	1.4625 ms	0.3992 ms	0.7692 ms	0.4074 ms	0.9691 ms
100×100	5	10.6593 ms	0.6735 ms	5.0648 ms	0.6937 ms	1.5499 ms
500×500	5	1167.3600 ms	13.9877 ms	17.7575 ms	15.6578 ms	35.1272 ms

Table 7: Running time measurements for Scenario 1

Grid Size	Keywords	Brute Force	Rabin-Karp	KMP	Boyer-Moore	Aho-Corasick
10×10	5	2,520	18	3,172	236	0
50×50	5	231,400	78	78,740	1,767	0
100×100	5	1,836,800	986	282,803	22,880	0
500×500	5	216,796,000	1,448	6,669,643	124,220	0

Table 8: Number of comparisons for Scenario 1

Scenario 2: Fixed Grid Size (100×100) with Varying Number of Keywords

Grid Size	Keywords	Brute Force	Rabin-Karp	KMP	Boyer-Moore	Aho-Corasick
100×100	1	1.8014 ms	0.1474 ms	0.2855 ms	0.1188 ms	1.6196 ms
100×100	10	20.1295 ms	1.1338 ms	2.0147 ms	1.1114 ms	2.0038 ms
100×100	50	99.2332 ms	5.4070 ms	9.4388 ms	5.1438 ms	3.5454 ms
100×100	100	195.2980 ms	11.0356 ms	49.1021 ms	10.8136 ms	4.8826 ms

Table 9: Running time measurements for Scenario 2

Grid Size	Keywords	Brute Force	Rabin-Karp	KMP	Boyer-Moore	Aho-Corasick
100×100	1	228,800	104	46,207	1,986	0
100×100	10	3,663,600	451	643,058	7,001	0
100×100	50	16,450,800	731	3,155,398	54,562	0
100×100	100	34,552,800	3,529	6,225,769	174,645	0

Table 10: Number of comparisons for Scenario 2

4.2 Performance Evaluation and Comments

Based on the experimental data presented above, the following observations and justifications address the core performance behaviors of the implemented algorithms.

- **Correlation with Big-O Complexity:**
  The growth rates observed in the experimental data heavily align with the theoretical Big-O complexities of the algorithms.
  - **Brute Force:** In Scenario 1, as the grid size increases from 10×10 to 500×500, the running time explodes from 0.0739 ms to 1167.36 ms, and comparisons reach over 216 million. This perfectly illustrates its $O(N \cdot M)$ worst-case complexity, where N is the text size and M is the pattern size. It must redundantly check every possible starting position.
  - **Aho-Corasick:** In Scenario 2, as the number of keywords increases from 1 to 100, Brute Force and KMP show linear growth in execution time. However, Aho-Corasick exhibits an incredibly flat growth curve (increasing only from 1.6196 ms to 4.8826 ms). This matches its theoretical complexity of $O(N + M + Z)$ (where Z is the number of matches). Because it builds an automaton, the search time relies primarily on the length of the text, remaining largely independent of the number of patterns being searched simultaneously.

- **Best Performers on Large Datasets vs. Small Datasets:**
  The "best" algorithm depends entirely on the nature of the dataset.
  - For a **large search space (Grid** 500×500**) with few keywords**, **Rabin-Karp** (13.9877 ms) and **Boyer-Moore** (15.6578 ms) performed the best. Boyer-Moore's efficiency is evident in its low comparison count $(124, 220)$, as its bad-character heuristic allows it to skip large sections of the text.
  - For a **large dictionary of keywords (100 keywords)**, **Aho-Corasick** is the absolute best performer (4.8826 ms), outperforming the next fastest (Boyer-Moore at 10.8136 ms) by more than double the speed.

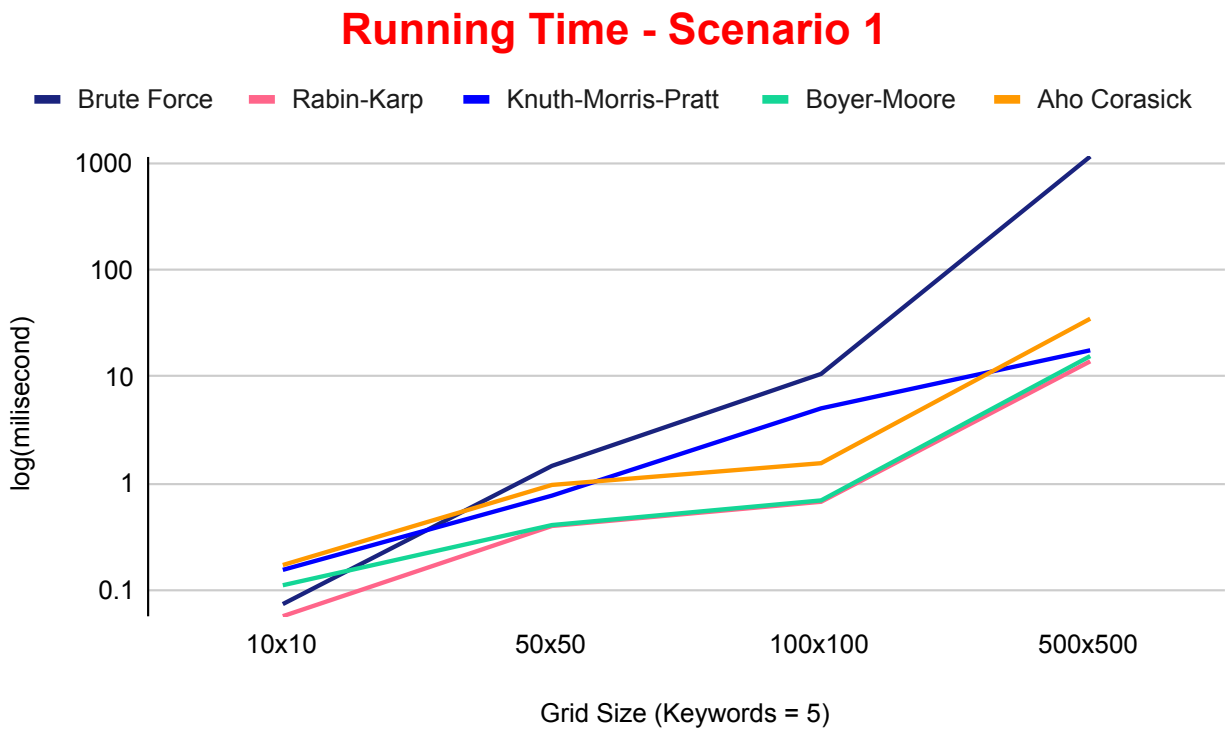
  The Brute Force Exception: There is a clear case where Brute Force was faster than the optimized approaches. In **Scenario 1, at the smallest grid size (**10×10**) with 5 keywords**, Brute Force (0.0739 ms) outperformed KMP (0.1547 ms), Boyer-Moore (0.1110 ms), and Aho-Corasick (0.1718 ms).

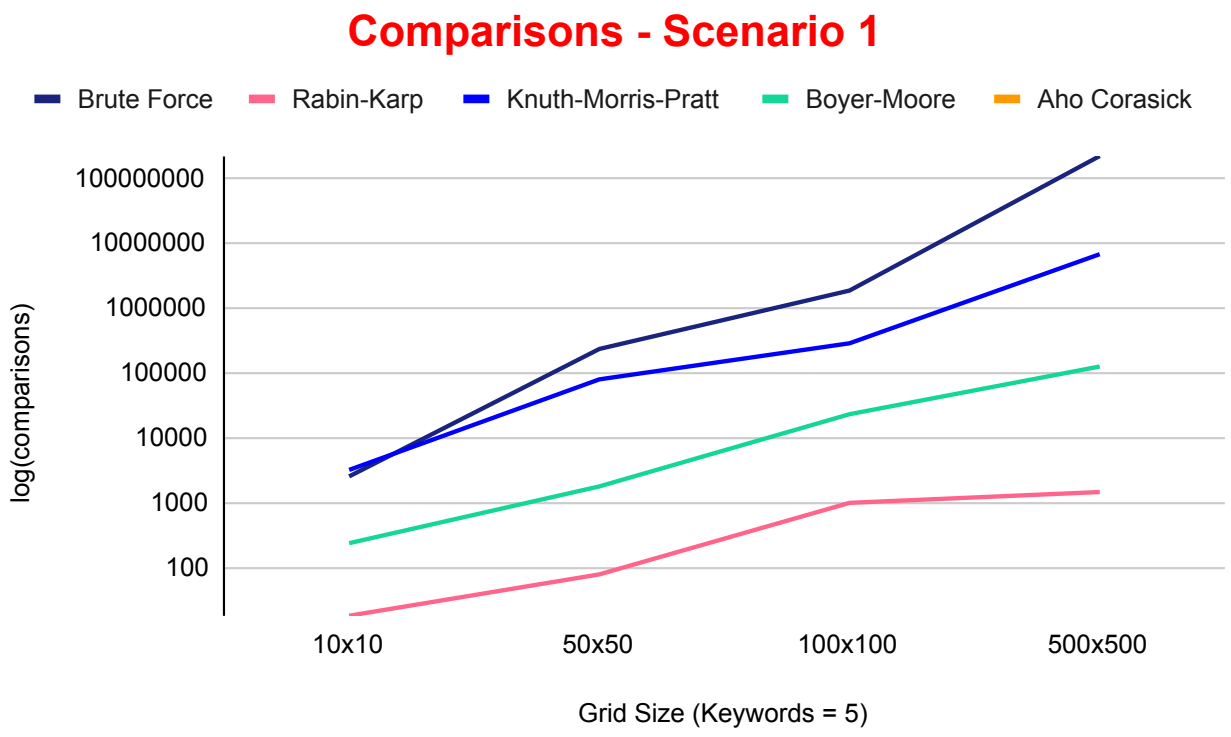
- **Justification of Results:**
  The phenomenon where Brute Force beats advanced algorithms on very small datasets is due to **preprocessing overhead**. Algorithms like KMP, Boyer-Moore, and Aho-Corasick require initial time to construct LPS arrays, shift tables, or complex automata before the search even begins. On a tiny 10×10 grid, the time taken to build these data structures exceeds the time it takes for Brute Force to simply execute nested loops to check the few available characters.

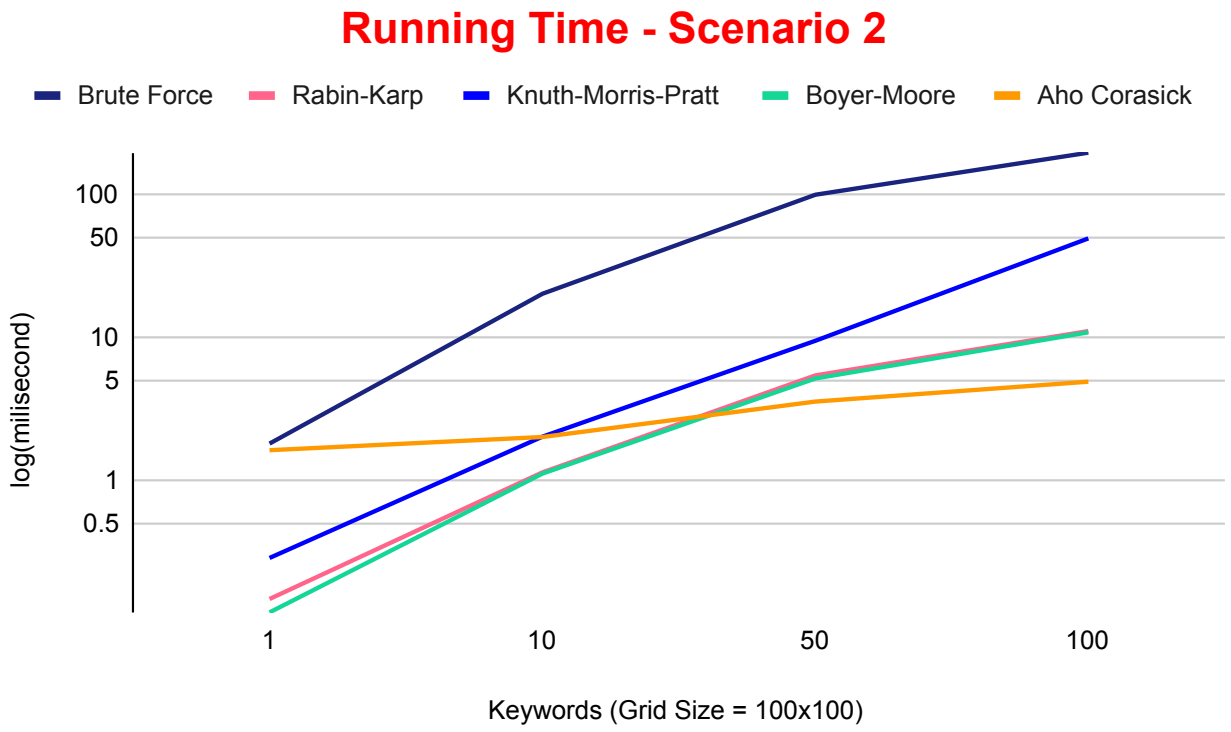
  Conversely, on large datasets, this preprocessing overhead pays off immensely. Rabin-Karp shows an astonishingly low number of comparisons (e.g., only 1,448 comparisons for a 500×500 grid) because it compares integer hash values rather than full strings, only resorting to character comparisons when a hash collision occurs. Meanwhile, Aho-Corasick dominates the multiple-keyword scenario because its automaton processes the grid exactly

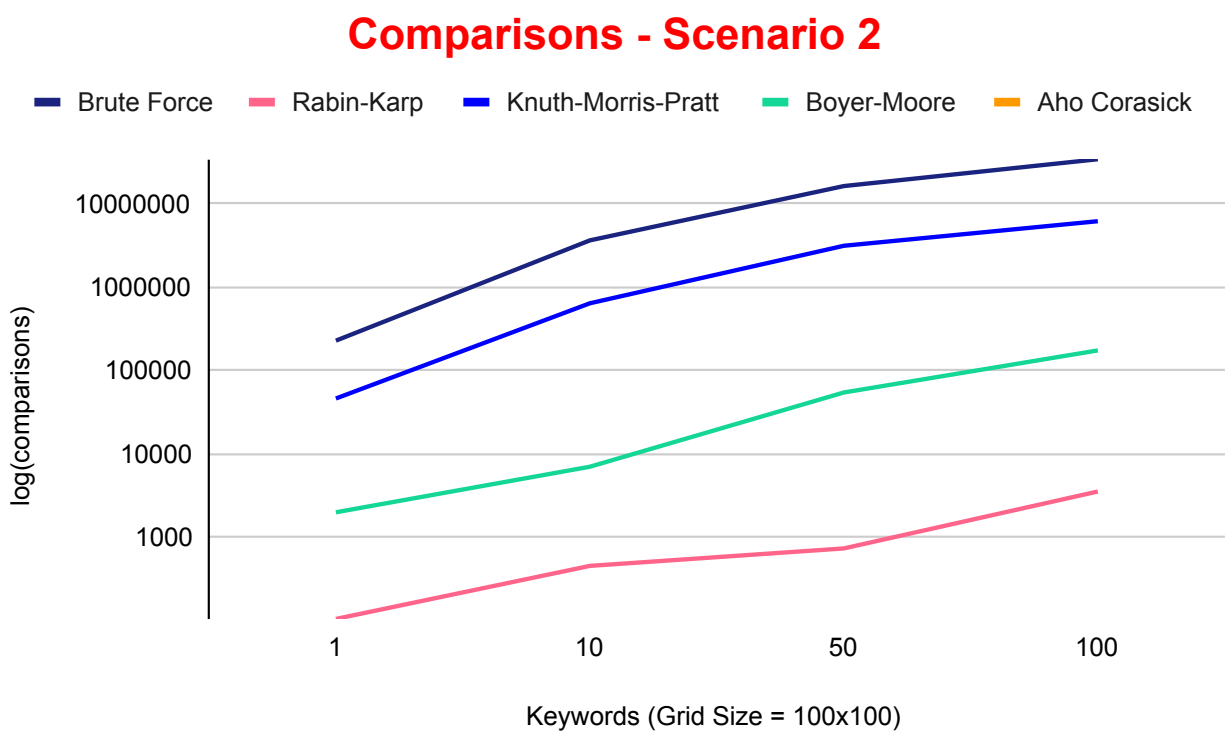
once, finding all 100 keywords simultaneously, whereas all other algorithms must iterate through the grid repeatedly for every single keyword.

4.3 Graphs


Running Time - Scenario 1
Brute Force
Rabin-Karp
Knuth-Morris-Pratt
Boyer-Moore
Aho Corasick
1000
100
10
1
0.1
log(milisecond)
10x10
50x50
100x100
500x500
Grid Size (Keywords = 5)


Comparisons - Scenario 1
Brute Force
Rabin-Karp
Knuth-Morris-Pratt
Boyer-Moore
Aho Corasick
100000000
10000000
1000000
100000
10000
1000
100
log(comparisons)
10x10
50x50
100x100
500x500
Grid Size (Keywords = 5)


Running Time - Scenario 2
Brute Force
Rabin-Karp
Knuth-Morris-Pratt
Boyer-Moore
Aho Corasick
log(milisecond)
100
50
10
5
1
0.5
1
10
50
100
Keywords (Grid Size = 100x100)


Comparisons - Scenario 2
Brute Force
Rabin-Karp
Knuth-Morris-Pratt
Boyer-Moore
Aho Corasick
log(comparisons)
10000000
1000000
100000
10000
1000
1
10
50
100
Keywords (Grid Size = 100x100)

5 Acknowledgements

First and foremost, the authors express our sincere gratitude to Le Thanh Tung & Tran Hoang Quan for giving guidance.

In addition, we are deeply grateful to VNOI wiki, Google Scholar and Introduction to Algorithms (CLRS) book for providing us with the essential materials, papers, source codes and examples that served as the foundation for this project.

Large Language Models also help us understand a few concepts when we were stuck and give us some useful suggestions to improve certain aspects of our work.

Lastly, we appreciate all members of Group 07 for their effort, consistency, hard work, and collaborative spirit throughout this project.

6 References

References

[1] Rawan Ali Abdeen. An algorithm for string searching based on brute-force algorithm. *IJCSNS International Journal of Computer Science and Network Security*, 11(7):24–27, 2011.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4th edition, 2022.

[3] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT press, 4th edition, 2022.

[4] Nadia El-Mabrouk and Maxime Crochemore. Boyer-moore strategy to efficient approximate string matching. *Lecture Notes in Computer Science*, October 1996.

[5] Saqib Iqbal Hakak, Amirrudin Kamsin, Palaiahnakote Shivakumara, Gulshan Amin Gilkar, Wazir Zada Khan, and Muhammad Imran. Exact string matching algorithms: survey, issues, and future research directions. *IEEE access*, 7:69614–69637, 2019.